SatQuery AI — Phase 1 Backend API Implementation Guide

Owner: Leo
Module: `backend/`
Framework: FastAPI
Base URL: `/api/v1`

---

1. Purpose of the Backend

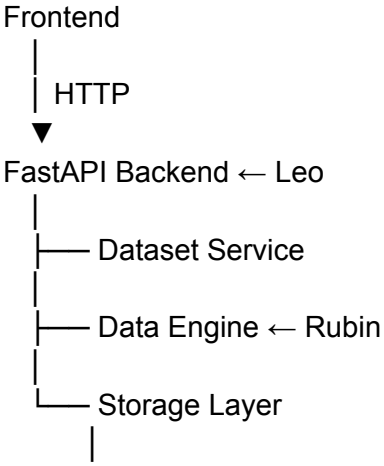
The backend is the **API boundary** between the frontend and SatQuery's internal modules.

Leo's backend should:

- receive requests from the frontend
- validate API requests
- call the appropriate internal service/module
- return standardized responses
- handle errors
- expose dataset and file APIs

The backend should **not** directly implement satellite processing, ML inference, or map logic.

Architecture



```
├── MongoDB
└── GridFS
```

---

2. API Base Path

All Phase 1 endpoints use:

/api/v1

Therefore:

```
GET    /api/v1/health
POST   /api/v1/datasets
GET    /api/v1/datasets
GET    /api/v1/datasets/{dataset_id}
DELETE /api/v1/datasets/{dataset_id}

POST   /api/v1/datasets/{dataset_id}/files
GET    /api/v1/datasets/{dataset_id}/files/{file_id}
DELETE /api/v1/datasets/{dataset_id}/files/{file_id}
```

---

3. Endpoint 1 — Create Dataset

Endpoint

POST /api/v1/datasets

Purpose

Create a **dataset record**.

This endpoint does **not upload the satellite file**.

Think of it as creating the dataset/container first.

Request

Headers

Content-Type: application/json

Body

```
{
  "name": "Chennai Flood Study",
  "dataset_type": "temporal"
}
```

Allowed `dataset_type`

single
temporal
multimodal

Backend Logic

Request
  ↓
FastAPI
  ↓
Pydantic validation
  ↓
Validate name + dataset_type
  ↓
Create dataset
  ↓
MongoDB
  ↓
Return dataset_id

Example MongoDB document:

```
{
  "_id": "...",
  "name": "Chennai Flood Study",
  "dataset_type": "temporal",
  "observations": [],
  "files": [],
  "processing": {
    "status": "uploading",
    "created_at": "...",
    "updated_at": "..."
```

```
  }
}
```

Response

Success

201 Created

```
{
  "dataset_id": "ds_001",
  "name": "Chennai Flood Study",
  "dataset_type": "temporal",
  "processing": {
    "status": "uploading"
  }
}
```

The most important value is:

dataset_id = ds_001

The frontend will use this ID when uploading files.

---

4. Endpoint 2 — List Datasets

Endpoint

GET /api/v1/datasets

Purpose

Return the datasets available to the frontend.

Request

No request body.

GET /api/v1/datasets

Backend Logic

Frontend
  ↓
GET /datasets
  ↓
FastAPI
  ↓
Storage Layer
  ↓
MongoDB
  ↓
Get datasets
  ↓
FastAPI
  ↓
Frontend

Response

200 OK


```
{
  "items": [
    {
      "dataset_id": "ds_001",
      "name": "Chennai Flood Study",
      "dataset_type": "temporal",
      "processing": {
        "status": "validated"
      }
    },
    {
      "dataset_id": "ds_002",
      "name": "Mumbai Urban Area",
      "dataset_type": "single",
      "processing": {
        "status": "validated"
      }
    }
  ],
  "pagination": {
```

```
    "page": 1,
    "page_size": 20,
    "total": 2
  }
}
```

Use our common `Pagination` structure.

---

5. Endpoint 3 — Get Dataset

Endpoint

GET /api/v1/datasets/{dataset_id}

Purpose

Retrieve one complete dataset.

Example:

GET /api/v1/datasets/ds_001

Backend Logic

Request
  ↓
Validate dataset_id
  ↓
Storage Layer
  ↓
MongoDB
  ↓
Find dataset
  ↓
Return dataset

Success Response

200 OK

```
{
  "dataset_id": "ds_001",
  "name": "Chennai Flood Study",
  "dataset_type": "temporal",

  "observations": [
    {
      "observation_id": "obs_001",
      "modality": "optical"
    }
  ],

  "files": [
    {
      "gridfs_id": "...",
      "filename": "chennai_before.tif",
      "content_type": "image/tiff",
      "size_bytes": 458000000
    }
  ],

  "processing": {
    "status": "validated"
  }
}
```


---

6. Endpoint 4 — Delete Dataset

Endpoint

DELETE /api/v1/datasets/{dataset_id}

Purpose

Delete a dataset and its associated files/references.

Backend Logic

Request
  ↓

Validate dataset_id
↓
Find dataset
↓
Delete associated GridFS files
↓
Delete MongoDB dataset
↓
Return success

The storage logic should be handled through the **Storage Layer**, rather than scattering MongoDB/GridFS operations throughout the API code.

Success Response

204 No Content

No response body is required.

---

7. Endpoint 5 — Upload File

Endpoint

POST /api/v1/datasets/{dataset_id}/files

⭐ This is the most important endpoint connecting Leo and Rubin.

This endpoint receives the actual satellite file.

Request

Content type

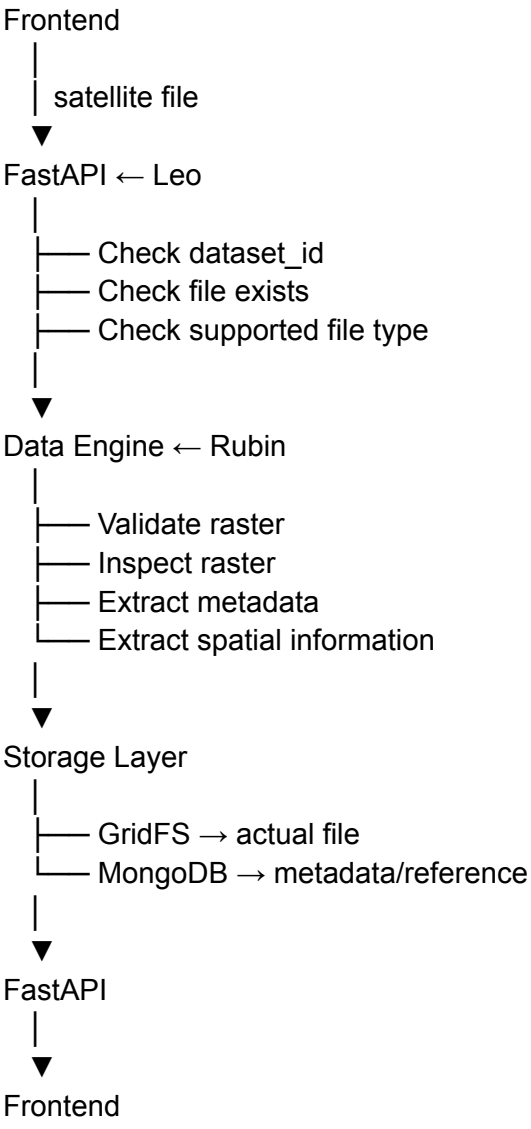
multipart/form-data

Example:

dataset_id = ds_001

file = chennai_before.tif

Backend Logic



Leo's Responsibility

Leo handles:

✓ HTTP request
✓ dataset_id validation
✓ file presence
✓ API-level file checks
✓ calling Data Engine
✓ handling Data Engine errors
✓ calling Storage Layer
✓ response formatting

Rubin's Responsibility

Rubin handles:

✓ Raster validation
✓ Raster inspection
✓ Metadata extraction
✓ Spatial information extraction
✓ Data Engine result

Example

User uploads:

sentinel_chennai.tif

Rubin might return:


```
{
  "valid": true,
  "raster": {
    "width": 10980,
    "height": 10980,
    "bands": 4,
    "resolution": 10,
    "crs": "EPSG:32644"
  },
  "spatial": {
    "bounds": "..."
  },
  "acquisition": {
    "datetime": null
```

```
  }
}
```

Notice:

acquisition.datetime = null

if the file doesn't contain that information.

Never invent missing metadata.

Success Response

201 Created

```
{
  "dataset_id": "ds_001",
  "file_id": "file_001",
  "status": "validated",
  "metadata": {
    "width": 10980,
    "height": 10980,
    "bands": 4,
    "resolution": 10,
    "crs": "EPSG:32644",
    "acquisition_datetime": null
  }
}
```

---

8. Endpoint 6 — Retrieve File

Endpoint

GET /api/v1/datasets/{dataset_id}/files/{file_id}

Purpose

Retrieve the actual uploaded file.

Logic

Frontend
↓
file_id
↓
FastAPI
↓
Storage Layer
↓
GridFS
↓
File stream
↓
Frontend

Leo should not load the entire large GeoTIFF unnecessarily into memory.

The implementation should support appropriate streaming behavior.

---

9. Endpoint 7 — Delete File

Endpoint

DELETE /api/v1/datasets/{dataset_id}/files/{file_id}

Purpose

Delete one file from a dataset.

Logic

Request
↓
FastAPI
↓
Validate dataset + file
↓
Storage Layer

↓
Delete GridFS file
↓
Remove file reference from dataset
↓
Response

Response

204 No Content

---

10. Endpoint 8 — Health

Endpoint

GET /api/v1/health

Purpose

Check that the backend is alive.

Response

200 OK

```
{
  "status": "ok"
}
```

For Phase 1, keep this simple.

---

11. Standard Error Response

All endpoints should follow one error format.

{

```
  "error": {
    "code": "DATASET_NOT_FOUND",
    "message": "Dataset was not found.",
    "details": null
  }
}
```

Phase 1 error codes

VALIDATION_ERROR
INVALID_DATASET_ID
DATASET_NOT_FOUND
FILE_NOT_FOUND
UNSUPPORTED_FILE_TYPE
INVALID_RASTER
PROCESSING_FAILED
STORAGE_ERROR
INTERNAL_ERROR

---

12. Example Complete Upload Scenario

This is the most useful thing for Leo to understand.

Step 1 — Create dataset

POST /api/v1/datasets

Request:

```
{
  "name": "Chennai Flood Study",
  "dataset_type": "temporal"
}
```

Response:

```
{
  "dataset_id": "ds_001",
  "name": "Chennai Flood Study",
  "dataset_type": "temporal",
  "processing": {
    "status": "uploading"
  }
```

}

Step 2 — Upload satellite file

POST /api/v1/datasets/ds_001/files

Request:

multipart/form-data

file = chennai_before.tif

Leo receives it.

Leo
↓
API validation
↓
Rubin

Rubin:

Read raster
↓
Validate
↓
Extract metadata
↓
Return structured result

Then:

Storage Layer
↓
GridFS → chennai_before.tif
MongoDB → metadata + reference

Response:


```
{
  "dataset_id": "ds_001",
  "file_id": "file_001",
  "status": "validated",
  "metadata": {
    "width": 10980,
    "height": 10980,
```

```
    "bands": 4,
    "resolution": 10,
    "crs": "EPSG:32644"
  }
}
```

---

13. What Leo Should NOT Implement

For Phase 1:

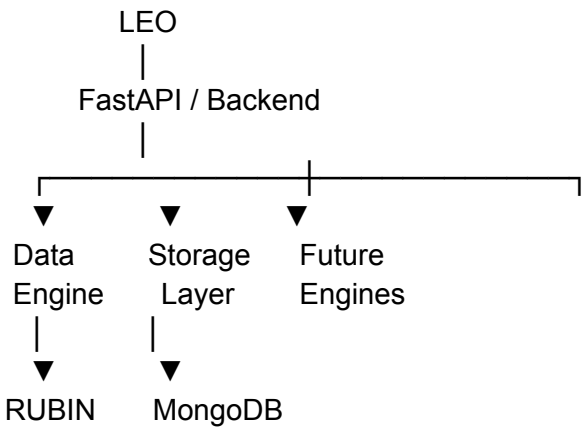
❌ Query Engine
❌ Natural-language processing
❌ Intent classification
❌ Task prediction
❌ AI model inference
❌ GeoChat
❌ CROMA
❌ Change detection
❌ SAR VQA
❌ Optical-SAR fusion
❌ Cross-attention

Those are later phases.

---

14. Final Responsibility Boundary

Leo should remember this:



Data Engine + GridFS

More precisely:

Leo:
"How does a request enter and leave SatQuery?"

Rubin:
"How do we validate and understand the satellite file?"

Storage:
"How do we save and retrieve it?"

Frontend:
"How do we present it to the user?"

---

15. Leo's Phase 1 Definition of Done

Leo's backend is ready when:

✓ FastAPI starts
✓ /health works
✓ Dataset can be created
✓ Dataset can be listed
✓ Dataset can be retrieved
✓ Dataset can be deleted
✓ File can be uploaded
✓ Data Engine can be called
✓ Validation result is handled
✓ Metadata result is handled
✓ File is stored through Storage Layer
✓ File can be retrieved
✓ File can be deleted
✓ Standard errors work
✓ Pydantic schemas match our common structures
✓ No direct frontend → database access
✓ No architecture boundary violations

Important: Leo should implement the endpoints according to the **existing API contracts and common data structures we already finalized**, rather than independently changing the

request/response formats. If he thinks a contract needs to change, bring it back to you as Tech Lead before changing it.